

YT-pipeline

February 8, 2026

1 YouTube Pipeline Implementation

1.1 Purpose

This notebook constructs a large-scale, analysis-ready dataset for studying YouTube virality by enriching a raw list of video URLs/IDs with creator-level, content-level, and engagement metadata from the YouTube Data API. The resulting dataset integrates textual metadata (titles, descriptions, tags), temporal features (video age, duration), creator social capital (subscriber counts, channel history), and engagement signals (views, likes, comments), enabling systematic investigation of which attributes are most strongly associated with viral outcomes.

1.2 Feature scope and design choices

In addition to standard metadata, the pipeline optionally extracts lightweight thumbnail image features (e.g., brightness, colorfulness, resolution) as coarse proxies for visual salience. These features are intentionally limited in complexity and are used only as exploratory controls rather than deep semantic representations. Raw thumbnail images are cached temporarily during processing and discarded after feature extraction to maintain a compact, reproducible dataset.

1.3 Methodological motivation

Consistent with prior work, this notebook prioritizes features that are observable at or near publication time (creator characteristics, metadata, visual proxies) while also retaining post-publication engagement metrics for comparative and explanatory analyses. This design reflects the central research objective: distinguishing predictors that plausibly contribute to virality from signals that primarily emerge after diffusion has already occurred.

1.4 Output dataset

The final output is a single enriched CSV file containing one row per YouTube video, identified by `video_id` (from the `youtube_data.csv`), and integrating technical, semantic, social, and engagement-related features. The dataset includes raw video and encoding attributes such as duration, resolution, bitrate, frame rate, and codec, alongside scraped metadata including category, URL, title, description, hashtags, and observed view, like, and comment counts. These fields are augmented with structured YouTube Data API features capturing publication timing (`yt_published_at`), standardized duration (`yt_duration_sec`), content category (`yt_category_id`), language metadata, live-broadcast and audience-designation flags, and validated engagement metrics (`yt_view_count`, `yt_like_count`, `yt_comment_count`). Creator-level social capital is represented through channel identifiers and attributes, including channel age,

country, subscriber count, total channel views, and upload volume. Finally, the dataset optionally includes lightweight, interpretable thumbnail-derived visual proxies—such as resolution, mean brightness, and colorfulness—computed from thumbnail URLs without retaining raw image data. Together, these columns produce an analysis-ready table that supports exploratory analysis, statistical modeling, and machine learning approaches for examining the relative influence of content design, creator reputation, and engagement dynamics on YouTube virality.

```
[16]: from dotenv import load_dotenv
import os

ENV_PATH = "/Users/maxchalekson/Northwestern University/Winter-2026/MSDS-422-0/
↳Final-Project/422-final-project/.env"

print("File exists?", os.path.exists(ENV_PATH))
loaded = load_dotenv(ENV_PATH, override=True)
print("dotenv loaded?", loaded)
print("Kernel sees key?", bool(os.getenv("YOUTUBE_API_KEY")))
```

```
File exists? True
dotenv loaded? True
Kernel sees key? True
```

```
[20]: from __future__ import annotations

import os
import re
import math
import time
import json
import hashlib
from dataclasses import dataclass
from typing import Optional, Dict, Any, List, Tuple

import pandas as pd
import requests
from tqdm.auto import tqdm # progress bars for notebook + terminal

# -----
# Project config (edit these)
# -----

PROJECT_DIR = "/Users/maxchalekson/Northwestern University/Winter-2026/
↳MSDS-422-0/Final-Project/422-final-project"
ENV_PATH = os.path.join(PROJECT_DIR, ".env") # should contain:
↳YOUTUBE_API_KEY=...
```

```

def ensure_youtube_api_key(env_path: str = ENV_PATH) -> str:
    key = os.getenv("YOUTUBE_API_KEY")
    if key:
        return key

    try:
        from dotenv import load_dotenv
        load_dotenv(env_path, override=True)
    except Exception:
        pass

    key = os.getenv("YOUTUBE_API_KEY")
    if key:
        return key

    raise ValueError(
        "Missing YouTube API key.\n"
        "Fix:\n"
        f" 1) Create {env_path}\n"
        "    with a line like: YOUTUBE_API_KEY=AIzaSy...\n"
        " 2) Run: pip install python-dotenv\n"
        " 3) Restart your Jupyter kernel / VS Code window\n"
        "Also ensure .env is in .gitignore so you don't leak your key."
    )

# -----
# Helpers: ID parsing + batching
# -----

_YT_ID_RE = re.compile(r"(?:v=|\/shorts\/|youtu\.be\/|\/embed\/
↪)([A-Za-z0-9_-]{11})")

def extract_video_id(url: str) -> Optional[str]:
    if not isinstance(url, str) or not url.strip():
        return None
    m = _YT_ID_RE.search(url)
    return m.group(1) if m else None

def chunked(lst: List[str], n: int) -> List[List[str]]:
    return [lst[i : i + n] for i in range(0, len(lst), n)]

def safe_int(x: Any) -> Optional[int]:
    try:

```

```

        if x is None:
            return None
        return int(x)
    except Exception:
        return None

# -----
# YouTube Data API client
# -----

@dataclass
class YouTubeAPI:
    api_key: str
    base_url: str = "https://www.googleapis.com/youtube/v3"
    session: Optional[requests.Session] = None
    sleep_s: float = 0.1

    def _sess(self) -> requests.Session:
        if self.session is None:
            self.session = requests.Session()
        return self.session

    def _get(self, path: str, params: Dict[str, Any]) -> Dict[str, Any]:
        params = dict(params)
        params["key"] = self.api_key
        url = f"{self.base_url}/{path}"
        r = self._sess().get(url, params=params, timeout=30)
        if r.status_code != 200:
            raise RuntimeError(f"YT API error {r.status_code}: {r.text[:500]}")
        time.sleep(self.sleep_s)
        return r.json()

    def videos_list(self, video_ids: List[str], parts: str) -> List[Dict[str, Any]]:
        out: List[Dict[str, Any]] = []
        batches = chunked(video_ids, 50)
        for batch in tqdm(batches, desc="Fetching video metadata",
                           unit="batch"):
            data = self._get("videos", {"part": parts, "id": ",".join(batch),
                                         "maxResults": 50})
            out.extend(data.get("items", []))
        return out

    def channels_list(self, channel_ids: List[str], parts: str) -> List[Dict[str, Any]]:
        out: List[Dict[str, Any]] = []

```

```

        batches = chunked(channel_ids, 50)
        for batch in tqdm(batches, desc="Fetching channel metadata",
↪unit="batch"):
            data = self._get("channels", {"part": parts, "id": ",".join(batch),
↪"maxResults": 50})
            out.extend(data.get("items", []))
        return out

# -----
# Feature extraction helpers
# -----

def iso8601_duration_to_seconds(dur: Optional[str]) -> Optional[int]:
    if not dur or not isinstance(dur, str):
        return None
    h = m = s = 0
    mobj = re.match(r"^PT(?:\d+)H?(?:\d+)M?(?:\d+)S?$", dur)
    if not mobj:
        return None
    if mobj.group(1): h = int(mobj.group(1))
    if mobj.group(2): m = int(mobj.group(2))
    if mobj.group(3): s = int(mobj.group(3))
    return h * 3600 + m * 60 + s

def pick_best_thumbnail(thumbnails: Dict[str, Any]) -> Tuple[Optional[str],
↪Optional[str]]:
    if not isinstance(thumbnails, dict):
        return (None, None)
    order = ["maxres", "standard", "high", "medium", "default"]
    for k in order:
        if k in thumbnails and isinstance(thumbnails[k], dict) and "url" in
↪thumbnails[k]:
            return thumbnails[k]["url"], k
    return (None, None)

def compute_image_features_from_url(img_url: Any, cache_dir: str) -> Dict[str,
↪Any]:
    feats: Dict[str, Any] = {
        "thumb_path": None,
        "thumb_sha1": None,
        "thumb_width": None,
        "thumb_height": None,
        "thumb_mean_brightness": None,
        "thumb_colorfulness": None,

```

```

}

# handle NaN / None / non-strings
if not isinstance(img_url, str) or not img_url.strip():
    return feats

os.makedirs(cache_dir, exist_ok=True)
fname = hashlib.sha1(img_url.encode("utf-8")).hexdigest() + ".jpg"
fpath = os.path.join(cache_dir, fname)

if not os.path.exists(fpath):
    try:
        r = requests.get(img_url, timeout=30)
        if r.status_code != 200:
            return feats
        with open(fpath, "wb") as f:
            f.write(r.content)
    except Exception:
        return feats

feats["thumb_path"] = fpath

try:
    from PIL import Image
    import numpy as np

    with open(fpath, "rb") as f:
        b = f.read()
    feats["thumb_sha1"] = hashlib.sha1(b).hexdigest()

    img = Image.open(fpath).convert("RGB")
    w, h = img.size
    feats["thumb_width"] = w
    feats["thumb_height"] = h

    arr = np.asarray(img).astype(np.float32)
    brightness = 0.2126 * arr[..., 0] + 0.7152 * arr[..., 1] + 0.0722 * arr[
↪arr[..., 2]
    feats["thumb_mean_brightness"] = float(np.mean(brightness))

    rg = arr[..., 0] - arr[..., 1]
    yb = 0.5 * (arr[..., 0] + arr[..., 1]) - arr[..., 2]
    std_rg = float(np.std(rg))
    std_yb = float(np.std(yb))
    mean_rg = float(np.mean(rg))
    mean_yb = float(np.mean(yb))
    feats["thumb_colorfulness"] = float(

```

```

        math.sqrt(std_rg**2 + std_yb**2) + 0.3 * math.sqrt(mean_rg**2 +
↪mean_yb**2)
    )
    except Exception:
        pass

    return feats

# -----
# Main enrichment function
# -----

def enrich_youtube_csv(
    csv_path: str,
    api_key: Optional[str] = None,
    video_id_col: str = "video_id",
    url_col: str = "url",
    out_csv_path: Optional[str] = None,
    add_thumbnail_features: bool = False,
    thumbnail_cache_dir: str = "thumb_cache",
) -> pd.DataFrame:
    api_key = api_key or ensure_youtube_api_key()

    df = pd.read_csv(csv_path)

    if video_id_col not in df.columns:
        df[video_id_col] = pd.NA

    if url_col in df.columns:
        missing_vid = df[video_id_col].isna() | (df[video_id_col].astype(str).
↪str.strip() == "")
        df.loc[missing_vid, video_id_col] = df.loc[missing_vid, url_col].
↪apply(extract_video_id)

    df[video_id_col] = df[video_id_col].astype(str).strip()
    df.loc[df[video_id_col].isin(["", "nan", "None"]), video_id_col] = pd.NA

    video_ids = df[video_id_col].dropna().unique().tolist()
    if not video_ids:
        raise ValueError("No valid video IDs found in the CSV (either in
↪video_id_col or parsed from url_col).")

    yt = YouTubeAPI(api_key=api_key)

    # video + channel metadata

```

```

video_items = yt.videos_list(video_ids,
↳parts="snippet,contentDetails,statistics,status")
video_rows: Dict[str, Dict[str, Any]] = {}

for item in video_items:
    vid = item.get("id")
    snippet = item.get("snippet", {}) or {}
    stats = item.get("statistics", {}) or {}
    content = item.get("contentDetails", {}) or {}
    status = item.get("status", {}) or {}

    thumb_url, thumb_quality = pick_best_thumbnail(snippet.
↳get("thumbnails", {}))

    video_rows[vid] = {
        "yt_title": snippet.get("title"),
        "yt_description": snippet.get("description"),
        "yt_published_at": snippet.get("publishedAt"),
        "yt_duration_sec": iso8601_duration_to_seconds(content.
↳get("duration")),
        "yt_category_id": snippet.get("categoryId"),
        "yt_tags_json": json.dumps(snippet.get("tags")) if snippet.
↳get("tags") is not None else None,
        "yt_default_language": snippet.get("defaultLanguage"),
        "yt_default_audio_language": snippet.get("defaultAudioLanguage"),
        "yt_made_for_kids": status.get("madeForKids"),
        "yt_live_broadcast_content": snippet.get("liveBroadcastContent"),
        "yt_view_count": safe_int(stats.get("viewCount")),
        "yt_like_count": safe_int(stats.get("likeCount")),
        "yt_comment_count": safe_int(stats.get("commentCount")),
        "yt_channel_id": snippet.get("channelId"),
        "yt_thumb_url": thumb_url,
        "yt_thumb_quality": thumb_quality,
    }

video_enriched = pd.DataFrame.from_dict(video_rows, orient="index")
video_enriched.index.name = video_id_col
df = df.merge(video_enriched, how="left", left_on=video_id_col,
↳right_index=True)

channel_ids = sorted(set([c for c in df["yt_channel_id"].dropna().tolist()
↳if isinstance(c, str)]))
if channel_ids:
    channel_items = yt.channels_list(channel_ids,
↳parts="snippet,statistics")
    channel_rows: Dict[str, Dict[str, Any]] = {}

```

```

for item in channel_items:
    cid = item.get("id")
    snippet = item.get("snippet", {}) or {}
    stats = item.get("statistics", {}) or {}
    channel_rows[cid] = {
        "yt_channel_title": snippet.get("title"),
        "yt_channel_published_at": snippet.get("publishedAt"),
        "yt_channel_country": snippet.get("country"),
        "yt_subscriber_count": safe_int(stats.get("subscriberCount")),
        "yt_channel_view_count": safe_int(stats.get("viewCount")),
        "yt_channel_video_count": safe_int(stats.get("videoCount")),
    }

channel_enriched = pd.DataFrame.from_dict(channel_rows, orient="index")
channel_enriched.index.name = "yt_channel_id"
df = df.merge(channel_enriched, how="left", on="yt_channel_id")

# -----
# Efficient thumbnail features: unique URLs only
# -----
if add_thumbnail_features:
    thumb_series = df["yt_thumb_url"].fillna("").astype(str).map(str.strip)

    unique_urls = [u for u in thumb_series.unique().tolist() if u]
    url_to_feats: Dict[str, Dict[str, Any]] = {}

    for url in tqdm(unique_urls, desc="Processing unique thumbnails",
        ↪unit="image"):
        url_to_feats[url] = compute_image_features_from_url(url,
        ↪thumbnail_cache_dir)

    # map back to rows (fast)
    feats_df = thumb_series.map(lambda u: url_to_feats.get(u, {
        "thumb_path": None,
        "thumb_sha1": None,
        "thumb_width": None,
        "thumb_height": None,
        "thumb_mean_brightness": None,
        "thumb_colorfulness": None,
    })).apply(pd.Series)

    df = pd.concat([df.reset_index(drop=True), feats_df.
        ↪reset_index(drop=True)], axis=1)

if out_csv_path:
    df.to_csv(out_csv_path, index=False)

```

```

return df

# -----
# Example run
# -----
if __name__ == "__main__":
    INPUT_CSV = os.path.join(PROJECT_DIR, "youtube_data.csv")
    OUTPUT_CSV = os.path.join(PROJECT_DIR, "youtube_data_enriched.csv")
    THUMB_DIR = os.path.join(PROJECT_DIR, "thumb_cache")

    df_enriched = enrich_youtube_csv(
        csv_path=INPUT_CSV,
        video_id_col="video_id",
        url_col="url",
        out_csv_path=OUTPUT_CSV,
        add_thumbnail_features=True,
        thumbnail_cache_dir=THUMB_DIR,
    )

    print(df_enriched.shape)
    print(df_enriched.columns.tolist())

```

```

Fetching video metadata: 100%|      | 352/352 [02:01<00:00, 2.90batch/s]
Fetching channel metadata: 100%|    | 347/347 [01:18<00:00, 4.43batch/s]
Processing unique thumbnails: 100%|  | 17491/17491 [1:05:37<00:00,
4.44image/s]

```

```

(17589, 45)
['video_id', 'duration', 'bitrate', 'bitrate(video)', 'height', 'width', 'frame
rate', 'frame rate(est.)', 'codec', 'category', 'url', 'title', 'description',
'hashtags', 'views', 'likes', 'comments', 'yt_title', 'yt_description',
'yt_published_at', 'yt_duration_sec', 'yt_category_id', 'yt_tags_json',
'yt_default_language', 'yt_default_audio_language', 'yt_made_for_kids',
'yt_live_broadcast_content', 'yt_view_count', 'yt_like_count',
'yt_comment_count', 'yt_channel_id', 'yt_thumb_url', 'yt_thumb_quality',
'yt_channel_title', 'yt_channel_published_at', 'yt_channel_country',
'yt_subscriber_count', 'yt_channel_view_count', 'yt_channel_video_count',
'thumb_path', 'thumb_sha1', 'thumb_width', 'thumb_height',
'thumb_mean_brightness', 'thumb_colorfulness']

```

```
[ ]:
```